

Documentación Uso IA

Pregunta Inicial

Analiza el pdf de los requerimientos para que armemos un plan, necesito que me expliques el paso a paso de lo que se debe de hacer para la realización del trabajo

Worked for 1m >

A continuación te presento el análisis completo y detallado del examen junto con el plan de trabajo paso a paso para abordarlo y completarlo con éxito.

Paso 1: Infraestructura y Repositorio

1. Crear el repositorio privado en GitHub: `tadb202620_examen_02` y agregar como colaborador a `jdrodas`.
2. Confirmar qué motor de base de datos usarás (por ejemplo PostgreSQL en Docker, MySQL o SQL Server).
3. Conectar el IDE (DBeaver, DataGrip o el de tu preferencia).

Paso 2: Modelado Relacional y Diagrama

1. Diseñar el diagrama ERD con las 5 tablas normalizadas.
2. Exportar la imagen en formato PNG/JPEG con fondo blanco.

Paso 3: Script DDL (`modelo.sql`) y Carga de Datos (ETL)

1. Escribir el script SQL con:
 - Creación de usuario/esquema y permisos mínimos.
 - `CREATE TABLE` con tipos de datos óptimos, PKs, FKs, constraints.
2. Cargar la sábana CSV (usando tabla temporal o script) y poblar mediante `INSERT INTO ... SELECT DISTINCT` las 5 tablas sin duplicados y manteniendo la integridad referencial.

Paso 4: Construcción de las Consultas SQL

1. Escribir la consulta de la Etapa 3 asegurando los filtros (`fecha_deteccion`, `severidades`, `COUNT(DISTINCT)`).
2. Escribir la consulta de la Etapa 4 para un usuario de prueba representativo.
3. Verificar que los resultados sean lógicos y consistentes con los datos.

Paso 5: Planes de Ejecución y Performance Tuning

1. Ejecutar `EXPLAIN ANALYZE` (o Execution Plan gráfico en el IDE) de cada consulta sin índices adicionales (solo las PKs).
2. Capturar el plan inicial: costo estimado, costo real, tiempo de ejecución y árbol de operaciones.
3. Diseñar e implementar los índices de optimización (índices B-Tree compuestos, índices para joins y filtros).
4. Volver a ejecutar el plan de ejecución con los índices creados.
5. Capturar la evidencia de la drástica reducción de costo/tiempo.

Paso 6: Compilación de Documentación

1. Redactar el documento Word (`.docx`) con los análisis de los planes de ejecución, capturas del árbol, costos y registros devueltos.
2. Preparar el PDF de infraestructura (o adjuntar el del Examen 1).
3. Generar el PDF con las capturas de la conversación con la IA.
4. Crear el `README.md` y el `.gitignore`.





Con base al análisis de los campos de los archivos csv y del enunciado, creé este modelo de datos

Necesito que me ayudes a verificar el modelo relacional que creé tras realizar un análisis del proyecto, realiza un análisis exhaustivo para realizarlo, no puede tener errores y debe de estar completamente normalizado hasta 3FN, solo quiero que te limites a eso, no cumplas con mas de esto, limitate a verificar el modelo de la base de datos, tambien tengo el script de creacion de las tablas .sql el cual esta en la carpeta raiz

Worked for 3m >

He realizado un análisis exhaustivo y riguroso de tu modelo relacional, confrontándolo tanto con el enunciado del examen en el PDF como con los datos reales de la sábana CSV (`sabana_brechas_seguridad_lote_1.csv`) y tu script `Creacion_Tablas.sql` .



Veredicto General

El modelo relacional es TOTALMENTE VÁLIDO, SÓLIDO y CUMPLE Estrictamente con la Tercera Forma Normal (3FN).

La arquitectura adoptada (5 tablas: 4 entidades fuertes + 1 tabla asociativa ternaria) responde con total precisión a lo solicitado por el docente en la página 6 del PDF.

1. Demostración Formal de Normalización (Hasta 3FN)

Primera Forma Normal (1FN)

- Condición: Atomicidad de los atributos, no existencia de grupos repetitivos ni atributos multivaluados, y existencia de una clave primaria definida.
- Cumplimiento:
 - Todos los atributos contienen valores atómicos y escalares.
 - No hay arrays ni concatenaciones de valores.
 - Cada tabla cuenta con una clave primaria formal.

Segunda Forma Normal (2FN)

- Condición: Estar en 1FN y que ningún atributo no clave dependa parcialmente de una subclave en tablas con clave primaria compuesta (dependencia funcional completa de la PK).
- Cumplimiento:
 - En `ORGANIZACION`, `TIPO_DATO`, `BRECHA` y `USUARIO`, las claves primarias son simples (`id_*`). Por definición, no pueden existir dependencias parciales en claves simples.
 - En `EXPOSICION_DATO`, la clave primaria es compuesta: (`id_usuario`, `id_brecha`, `id_tipo_dato`).
 - El único atributo no clave es `fecha_notificacion`. Al analizar la sábana de datos, se verifica que un mismo usuario en una misma brecha puede tener fechas de notificación diferentes según el tipo de dato expuesto (se evidencian 370 casos en los datos). Por ende, `fecha_notificacion` depende funcionalmente de la terna completa PK → `fecha_notificacion`, garantizando que no hay dependencia parcial. Cumple estrictamente 2FN.

Tercera Forma Normal (3FN)

- Condición: Estar en 2FN y que ningún atributo no clave dependa transitivamente de la clave primaria (es decir, $X \rightarrow Y \rightarrow Z$ donde Y no sea clave candidata).
- Cumplimiento tabla por tabla:
 - `ORGANIZACION`: `sector`, `pais` y `tanamo_empleados` dependen exclusivamente de `id_organizacion` (o de `nombre_organizacion`, que es clave candidata única). No existe dependencia entre `pais` y `sector`, ni entre `tanamo_empleados` y `pais`.
 - `TIPO_DATO`: `categoria_sensibilidad` depende directamente del tipo de dato (`id_tipo_dato` / `nombre_tipo_dato`).
 - `BRECHA`: `fecha_ocurrencia`, `fecha_deteccion`, `vector_ataque`, `severidad`, `registros_afectados` y `costo_estimado` son atributos propios del incidente, dependiendo directamente de `id_brecha` / `codigo_brecha`.
 - `USUARIO`: `pseudonimo`, `email_hash`, `pais_residencia` y `fecha_registro` dependen directamente del usuario. El país de residencia del usuario es independiente del país de la organización donde está registrado (comprobado en los datos: clientes de múltiples nacionalidades por organización).
 - `EXPOSICION_DATO`: No contiene atributos no clave intermedios que generen transitividad.

Quiero hacer la inserción de los datos mediante una tabla intermedia la cual ya tengo creada

```
CREATE TABLE stg_sabana
(
  nombre_organizacion    varchar(100),
  sector_organizacion    varchar(50),
  pais_organizacion      varchar(50),
  tamano_empleados_organizacion int,
  codigo_brecha          char(8),
  fecha_ocurrencia       date,
  fecha_deteccion        date,
  vector_ataque          varchar(60),
  severidad_incidente    varchar(10),
  registros_afectados_total int,
  costo_estimado_total   decimal(15, 2),
  codigo_usuario         char(9),
  pseudonimo_usuario     varchar(50),
  email_hash_usuario     char(32),
  pais_residencia_usuario varchar(50),
  fecha_registro_usuario date,
  tipo_dato_expuesto     varchar(50),
  categoria_sensibilidad_dato varchar(10),
  fecha_notificacion_usuario date
);

INSERT INTO organizacion (nombre_organizacion, sector, pais, tamano_empleados)
SELECT DISTINCT nombre_organizacion, sector_organizacion, pais_organizacion, tamano_empleados_organizacion
FROM stg_sabana;
```

Quiero que me ayudes a terminar de hacer la inserción desde esta tabla hacia las demás es engorroso a mano

Thought for 22s

Para realizar la inserción hacia las tablas normalizadas, el orden debe respetar estrictamente la integridad referencial (primero las tablas maestras/padres y finalmente las tablas hijas y la tabla asociativa).

Aquí tienes el script SQL completo con los `INSERT INTO ... SELECT` optimizados y con sus respectivos `JOIN` para resolver las llaves foráneas (`id_*`):

```
sql

-- =====
-- SCRIPT DE CARGA Y POBLACIÓN DESDE LA TABLA STAGING (stg_sabana)
-- =====

-- -----
-- 1. POBLAR TABLA: organizacion
-- (No depende de ninguna tabla)
-- -----

INSERT INTO organizacion (nombre_organizacion, sector, pais, tamano_empleados)
SELECT DISTINCT
  nombre_organizacion,
  sector_organizacion,
  pais_organizacion,
  tamano_empleados_organizacion
FROM stg_sabana;

-- -----
-- 2. POBLAR TABLA: tipo_dato
-- (Catálogo fijo de 10 registros, no depende de ninguna tabla)
-- -----

INSERT INTO tipo_dato (nombre_tipo_dato, categoria_sensibilidad)
SELECT DISTINCT
  tipo_dato_expuesto,
  categoria_sensibilidad_dato
FROM stg_sabana;

-- -----
-- 3. POBLAR TABLA: brecha
-- (Depende de organizacion para obtener id_organizacion)
-- -----

INSERT INTO brecha (
  codigo_brecha,
  id_organizacion,
  fecha_ocurrencia,
  fecha_deteccion,
  vector_ataque,
  severidad,
  registros_afectados,
  costo_estimado
)
SELECT DISTINCT
  s.codigo_brecha,
  o.id_organizacion,
  s.fecha_ocurrencia,
  s.fecha_deteccion,
```

La creación del usuario con privilegios mínimos ya está, Quiero crear un índice sobre la tabla fecha que me ayude a no tener que leer toda la tabla brecha y me ayude a agilizar la consulta 3

Esta fue la consulta que diseñé

```
SELECT o.nombre_organizacion,
       b.codigo_brecha,
       b.fecha_deteccion,
       b.vector_ataque,
       COUNT(DISTINCT e.id_usuario) AS usuarios_afectados
FROM brecha b
     JOIN organizacion o ON o.id_organizacion = b.id_organizacion
     JOIN exposicion_datos e ON e.id_brecha = b.id_brecha
     JOIN tipo_datos t ON t.id_tipo_datos = e.id_tipo_datos
WHERE b.fecha_deteccion BETWEEN '2025-10-07' AND '2026-10-07'
     AND b.severidad IN ('Alta', 'Crítica')
     AND t.categoria_sensibilidad = 'Crítica'
GROUP BY b.id_brecha,
         o.nombre_organizacion,
         b.codigo_brecha,
         b.fecha_deteccion,
         b.vector_ataque
```

9:15 p. m.  

Worked for 1m >

Tu propuesta es **muy acertada** y va exactamente en la dirección correcta para *Performance Tuning*. Aquí tienes la evaluación técnica detallada de la consulta, el índice propuesto y la recomendación exacta para `exposicion_datos`.

1. Evaluación de la Consulta SQL (Etapa 3)

- **Sargabilidad (Excelente):** Al usar `b.fecha_deteccion BETWEEN '2025-10-07' AND '2026-10-07'` con literales en vez de envolver la columna en funciones como `YEAR(fecha_deteccion) = 2026`, permites que el motor utilice búsquedas por rango sobre índices B-Tree.



1. Evaluación de la Consulta SQL (Etapa 3)

- **Sargabilidad (Excelente):** Al usar `b.fecha_deteccion BETWEEN '2025-10-07' AND '2026-10-07'` con literales en vez de envolver la columna en funciones como `YEAR(fecha_deteccion) = 2026`, permites que el motor utilice búsquedas por rango sobre índices B-Tree.
- **Filtros e Integridad con el Enunciado:** Cumple con los 4 requerimientos técnicos del docente (filtro de fechas, filtro por severidad y sensibilidad, joins y orden descendente por conteo).
- **Compatibilidad GROUP BY :** Todas las columnas del `SELECT` están explícitamente en el `GROUP BY`, evitando problemas con la directiva estricta `ONLY_FULL_GROUP_BY` de MySQL.

Esta consulta es para la etapa 4, recuerda que esta pide lo siguiente,

Etapa 4: Historial de exposición de un usuario específico. 30%

Un usuario reporta una solicitud de derecho de acceso y pide conocer, dado su código de usuario:

- en cuáles brechas de seguridad se vio afectado
- qué tipo de dato se expuso en cada una
- la categoría de sensibilidad de ese dato
- el nombre de la organización responsable
- la fecha en la que fue notificado

Esto es lo que tengo, se puede hacer de otra manera?

```
SELECT b.codigo_brecha,
       t.nombre_tipo_dato,
       t.categoria_sensibilidad,
       o.nombre_organizacion,
       e.fecha_notificacion
FROM usuario u
     JOIN exposicion_dato e ON e.id_usuario = u.id_usuario
     JOIN brecha b ON b.id_brecha = e.id_brecha
```

Thought for 23s >

Tu consulta actual es **completamente correcta y precisa**: cumple con todos los campos pedidos por el docente, la lógica relacional es impecable y la relación entre la organización y la brecha (`o.id_organizacion = b.id_organizacion`) es la correcta.

Respecto a si se puede hacer de otra manera, aquí tienes 3 alternativas técnicas válidas y cómo se comparan:

Alternativa 1: Subconsulta Escalar Directa (Muy recomendada para Tuning)

En lugar de unir la tabla `usuario` en la cadena de `JOIN`, resuelves primero el `id_usuario` mediante una subconsulta rápida sobre el índice `UNIQUE` de `codigo_usuario`:

```
sql
SELECT b.codigo_brecha,
       t.nombre_tipo_dato,
       t.categoria_sensibilidad,
       o.nombre_organizacion,
       e.fecha_notificacion
FROM exposicion_dato e
     JOIN brecha b ON b.id_brecha = e.id_brecha
     JOIN tipo_dato t ON t.id_tipo_dato = e.id_tipo_dato
     JOIN organizacion o ON o.id_organizacion = b.id_organizacion
WHERE e.id_usuario = (SELECT id_usuario FROM usuario WHERE codigo_usuario = 'US-006809')
ORDER BY e.fecha_notificacion ASC;
```

- **Ventaja técnica:** Elimina un `JOIN` del árbol de ejecución. El optimizador evalúa la subconsulta una sola vez (`const/scalar lookup`) y ataca directamente la tabla `exposicion_dato` usando el índice primario en `id_usuario`.

No se tomo Ninguna de las alternativas, preguntándole a Gemini desde el navegador me recomendó no hacerlo

La primera consulta (Consulta 1) es superior y está mejor estructurada.

A nivel de rendimiento en el motor de MySQL ambas ejecutan de forma casi idéntica, pero la Consulta 1 es más sólida tanto en su sintaxis SQL como en la determinación de sus resultados.

Diferencias principales

Criterio	Consulta 1 (JOIN)	Consulta 2 (Subconsulta)
Relación con <code>usuario</code>	Utiliza un <code>JOIN</code> u relacional estándar.	Utiliza una subconsulta escalar (<code>SELECT id_usuario ...</code>) en la cláusula <code>WHERE</code> .
Ordenamiento (<code>ORDER BY</code>)	Determinista: Ordena por <code>fecha_notificacion</code> , y desempatan <code>codigo_brecha</code> y <code>nombre_tipo_datos</code> .	No determinista: Solo ordena por <code>fecha_notificacion</code> . Si el usuario fue notificado de varios datos el mismo día, el orden interno será aleatorio.
Estilo y Legibilidad	Estándar en bases de datos relacionales cuando se cruzan múltiples tablas.	Menos convencional para selecciones simples de clave.

...

Análisis de optimización (¿Cuál ejecuta mejor?)

Ambas consultas se ejecutan con la misma alta eficiencia debido a la estructura de índices de tu base de datos:

- Búsqueda por `codigo_usuario`:** Como `codigo_usuario` tiene una restricción `UNIQUE` en la tabla `usuario`, MySQL realiza una búsqueda instantánea de tipo `const` (1 sola fila).
- Uso de la clave primaria agrupada:** Ambas obtienen el `id_usuario` e inmediatamente van a la tabla `exposicion_datos`. Dado que la clave primaria de `exposicion_datos` es (`id_usuario`, `id_brecha`, `id_tipo_datos`), MySQL busca directamente en el índice primario filtrando por el primer campo (`id_usuario`).